



SM40.COM

Vulnerability Assessment Report

February 2026

PREPARED BY

MIR MUHAMMED AHSAN ALI

CONTACT

@M999ALI

ali.codes1279@gmail.com



Limitations on Disclosure and Use of this Document

Unauthorized use or reproduction of this document is prohibited. The confidentiality and the disclosure of this report is being governed by the standards Non-Disclosure Agreement (NDA).

This report contains information about potential vulnerabilities of the digital assets of **Swiratul Mustaqeem** and methods for exploiting them. Special precautions must be taken to protect the confidentiality of both this document and the information contained herein. A copy of this report is submitted to Emon vai Robi as per the assessment is authorised by him.

By providing this report, the inviolable security of the assets are not being constituted by any form of presentation, warranty, or guarantee. While the applied methodology includes both automated and manual testing to identify the most common security issues.

Document Details

Title	Vulnerability Assessment Report
Project Resource	Mir Muhammed Ahsan Ali
Project Duration	06/02/2026 – 16/02/2026



Contents

Scope of the assessment-----	3
Assessment Findings-----	3
1. Clickjacking vulnerability found in [REDACTED].sm40.com-----	4
2. Reflected XSS found in [REDACTED].sm40.com-----	6
Important Notes-----	8
⚠ Chaining Vulnerabilities 1 & 2 – Stealing Credentials via Phishing-----	8
Conclusion-----	8



Scope of the assessment

*.sm40.com (wildcard)



Assessment Findings

Table of Status:

Status Name	Action
Reported	Vulnerability detected, needs action.
Risk Accepted	Risk acknowledged, but no fix applied.
Mitigated	Temporary fix applied.
Fixed	Vulnerability fully patched.
False Positive	Finding dismissed due to none existing threat

The findings are given below. Click on the findings to learn more.

Findings	Status
1. Clickjacking vulnerability found in [REDACTED].sm40.com	Reported
2. Reflected XSS found in [REDACTED].sm40.com	Reported

1. Clickjacking vulnerability found in [REDACTED].sm40.com

Severity: Medium

Status: Reported

Description

[REDACTED].sm40.com can be framed by external pages, allowing clickjacking attacks where users are tricked into clicking unintended elements (UI redress attack).

References

[Clickjacking | OWASP Foundation](#)

[CWE-1021: Improper Restriction of Rendered UI Layers or Frames \(4.19.1\)](#)

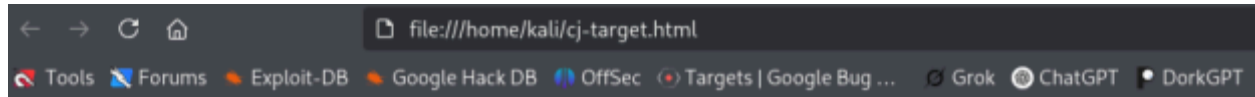
Proof of Concept

Click the link below to test the vulnerability right away:

[https://clickjacker.io/test?url=https://\[REDACTED\].sm40.com](https://clickjacker.io/test?url=https://[REDACTED].sm40.com)

```
<html>
  <head>
    <title>Clickjacking Test Page</title>
  </head>
  <body>
    <h1>Clickjacking Test Result<</h1>
    <h2>Target: <a href="https://[REDACTED].sm40.com/">https://[REDACTED].sm40.com/</a></h2>
    <h3>If you see the target website rendered below, you are vulnerable!</h3>
    <iframe width="900" height="600" src="https://[REDACTED].sm40.com/login" </iframe>
    <iframe style="position: absolute; left: 20px; top: 250px; opacity: 0.8; background: AliceBlue; font-weight: bold;" src="cj-attacker.html"></iframe>
  </body>
</html>
```

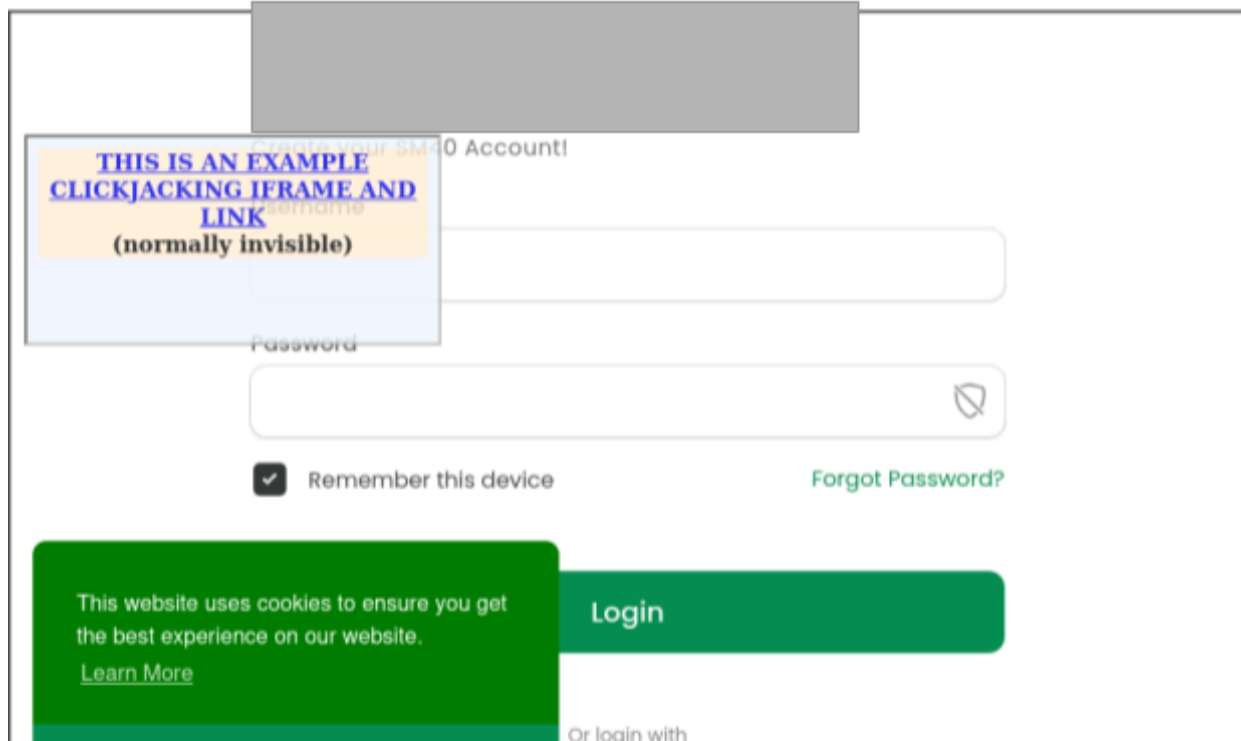
The iframe event creates a window to [REDACTED].sm40 login page into the hackers website due to lack of security headers.



Clickjacking Test Results

Target: [https://\[REDACTED\]](https://[REDACTED])

If you see the target website rendered below, it is **VULNERABLE**.



Remediation

Implement all missing headers, especially **X-Frame-Options: DENY** or **SAMEORIGIN** and a comprehensive **Content-Security-Policy**, to instruct browsers on security enforcement.

Read for more information about remediation:

[Clickjacking - Security | MDN](https://developer.mozilla.org/en-US/docs/Glossary/Clickjacking)

<https://dev.to/rigalpatel001/preventing-clickjacking-attacks-in-javascript-39pj/>

2. Reflected XSS found in [REDACTED].sm40.com

Severity: Medium to High

Status: Reported

Description

Reflected XSS vulnerability exists on [REDACTED].sm40.com due to improper neutralization of user inputs (name, mobile, email, quizID parameters). Malicious payloads in URL parameters are reflected back into the HTML response without escaping or sanitization, allowing execution of arbitrary JavaScript in the victim's browser (e.g., redirects or potential malware delivery).

References

[Cross Site Scripting \(XSS\) | OWASP Foundation](#)

[CWE-79: Improper Neutralization of Input During Web Page Generation \('Cross-site Scripting'\) \(4.19.1\)](#)

Proof of Concept

Redirect to attacker's site (demonstrates HTML code execution):

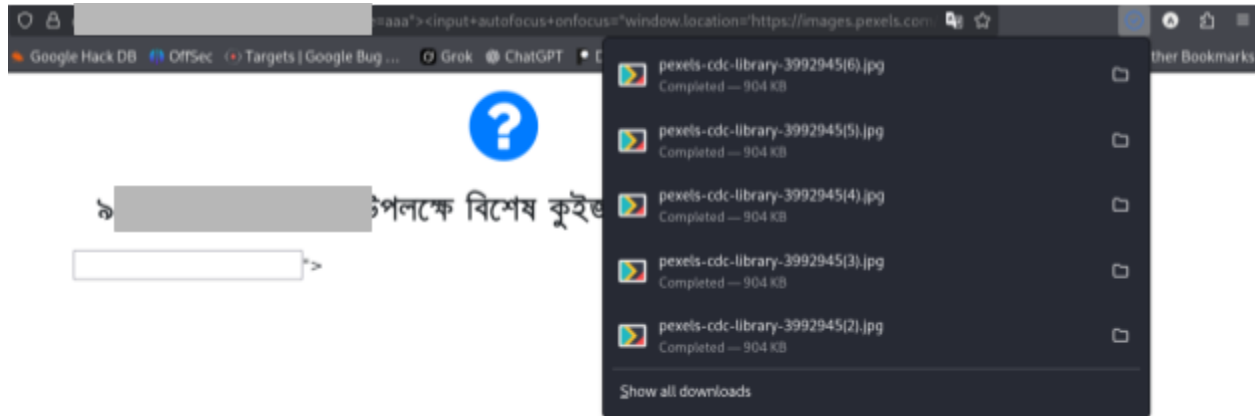
`https://[REDACTED].sm40.com/?[REDACTED]=aaa%22%3E%3Cinput+autofocus+onfocus=%22window.location=%27https://pranx.com/google-terminal/%27%22%3E`

This is because the browser **mistakes** the **attacker's** payload **for an** HTML snippet sent by the server and executes it.

```
<input autofocus onfocus="window.location='https://pranx.com/google-terminal/'">&email=aaa@aaa">
b1Btn" class="btn btn-success next-step"><i class="fa fa-arrow-left"></i> পূর্বের ধাপে নিয়ে যান </button>
```

This forces download of an image (used here as proof-of-concept for potential malware delivery via forced download of malicious files):

https://[REDACTED].sm40.com/?[REDACTED]=aaa%22%3E%3Cinput+autofocus+onfocus
 =%22window.location=%27https://images.pexels.com/photos/14000464/pexels-photo-1
 4000464.jpeg?cs=srgb%26dl=pexels-lucasandrade-14000464.jpg%26fm=jpg%27%22%3
 E



{Proof of downloading files without consent}

Remediation

- **Primary Fix:** Implement proper output encoding/escaping for all user-controlled data reflected in HTML (e.g., use HTML entity encoding for body content, attribute encoding for attributes).
- Add **Content-Security-Policy (CSP)** header with **default-src 'self'**; and restrict script-src to trusted sources.
- Set **X-XSS-Protection: 1; mode=block** (legacy defense-in-depth).
- Validate/sanitize input server-side (allow only expected characters for each field).
- Use modern frameworks/libraries with built-in escaping (e.g., if using PHP: htmlspecialchars()).

Apply fixes server-side, **re-test with the PoC payloads**, and confirm no execution occurs.

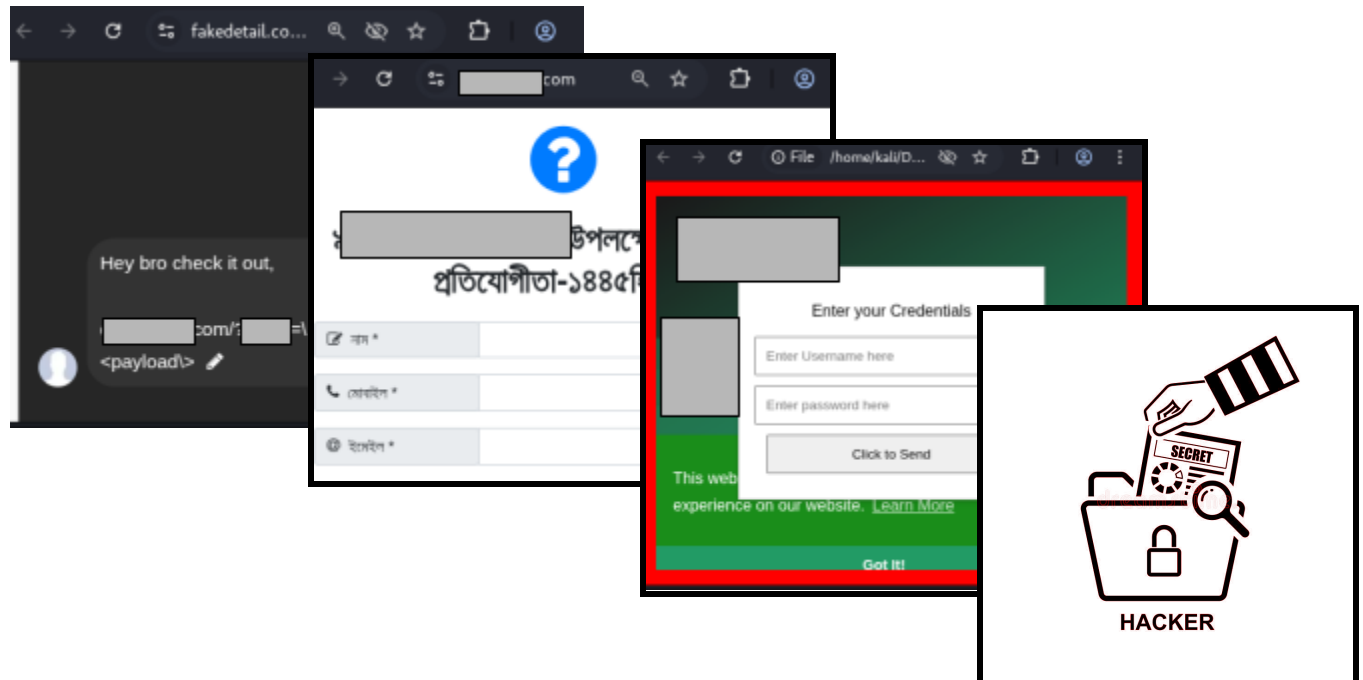


Important Notes

⚠ Chaining Vulnerabilities 1 & 2 – Stealing Credentials via Phishing

Attack Scenario:

An attacker can combine the **Reflected XSS** ([REDACTED].sm40.com) and **Clickjacking** ([REDACTED].sm40.com) to trick users into entering their login credentials on a fake page.



How it works (step by step):

1. Attacker creates a malicious link using the XSS vulnerability on [REDACTED].sm40.com that instantly redirects to their fake website.
2. On their fake site, they secretly load the real [REDACTED].sm40.com login page inside an invisible iframe (possible because of clickjacking).
3. They place fake email/password boxes exactly over the real ones in the iframe.
4. Victim receives the link (starts with [REDACTED].sm40.com/?[REDACTED]=...), thinks it's legitimate, and clicks it.
5. The page redirects to the attacker's fake site, which looks identical to the real [REDACTED].sm40.com login page.
6. Victim enters their email and password → attacker captures them.

Result: Victim unknowingly hands over his sm40 credentials.

Severity: High

(Two medium issues together become a serious credential-theft attack.)



Conclusion

This testing was based on the technologies and known threats as of the date of this document. All the security issues discovered during that exercise were analyzed and described in this report. Please note that as technologies and risks change over time, the vulnerabilities associated with the operation of systems described in this report, as well as the actions necessary to reduce the exposure to such vulnerabilities, will also change.